

Module 4: React + Express Codebase

Understanding

Explore a Myntra-style storefront to learn architecture reading, data flow tracing, and technical documentation.

Module focus

- Understand a full-stack React + Express codebase
- Trace frontend entry points and backend API routes
- Use prompts to summarize, document, and report findings

Core artifact

- Myntra-style store with Vite frontend and Express catalog API

What Learners Explore

The app gives learners a small but realistic codebase with visible frontend and backend responsibilities.

Frontend

- React/Vite single-page app
- Hero, live categories, product grid, and tech summary
- Status messaging for loading, ready, and error states

Backend

- Node/Express catalog service
- GET /api/products endpoint
- Curated products with formatted INR prices and delivery promises

Repository Map

Students learn to describe the codebase through directories, entry points, and runtime commands.

Key paths

- myntra-style-store/frontend/src/main.jsx
- myntra-style-store/frontend/src/App.jsx
- myntra-style-store/backend/index.js
- top-up-ideas/docs/*.md

Runtime commands

- Backend: npm install, then npm start on port 5174
- Frontend: npm install, then npm run dev on port 5173

First Pass: Inspect the Running Stack

The first pass grounds understanding in runtime behavior before documentation begins.

Learner actions

- Launch the Express backend
- Launch the Vite frontend
- Open `http://localhost:5173`
- Confirm the frontend fetches from `http://localhost:5174/api/products`

Discussion points

- How Express shapes the response
- How React stores banner, product, and status state
- How UI readiness is surfaced to the learner

Data Flow and API Contract

The central teaching moment is following data from server objects to rendered product cards.

Backend responsibilities

- Stores curated catalog objects
- Formats prices with Intl.NumberFormat en-IN
- Returns banner text and curated product array
- Enables CORS and JSON middleware

Frontend responsibilities

- Fetches /api/products in useEffect
- Stores banner and curated products in state
- Derives categories with useMemo
- Renders ProductCard components

Second Pass: Document the Codebase

Prompt docs turn exploration into reusable engineering knowledge.

Prompt suites

- codebase-understanding.md maps architecture and boundaries
- summary-prompts.md creates concise technical summaries
- documentary-prompts.md frames a technical story
- report-prompts.md records inspection, validation, and next steps

Output habit

- Keep summaries tied to files, commands, endpoints, and observed behavior

Architecture Storytelling

Module4 asks learners to explain not just what exists, but why the pieces collaborate.

Technical personas

- Designer: CSS, hero layout, responsive product cards
- Frontend engineer: React components, hooks, fetch state
- Backend engineer: Express route, formatter, catalog contract

Narrative arc

- Business need: festival-ready storefront
- Implementation: React UI plus Express catalog
- Evidence: running app, network response, and prompt-based docs

Third Pass: Extend the Experience

Learners apply one modification and then explain the technical impact.

Extension ideas

- Mega menu for Indian festivals
- Filterable product drawer
- Localized rotating offer banner
- Mini cart with localStorage
- Backend polish with JSON data, logging, or validation

After the change

- Update summaries and reports
- Record commands used for validation
- Call out tradeoffs and future work

Prompting Pattern for Codebase Understanding

Good prompts help Codex read like an engineer onboarding to a larger system.

Prompt recipe

- Name the entry points and scripts
- Ask for the data path from API to UI
- Request dependency boundaries by package.json
- Ask for operational notes: CORS, logging, errors
- Require file-backed claims and actionable summaries

Reusable principle

- Summaries should make the next engineer faster.

Wrap-Up Deliverables

The module ends with a technical story learners can share and improve.

Learners submit

- A codebase summary covering frontend, backend, and docs
- A data-flow explanation for `/api/products`
- One implemented or proposed extension
- A short report with validation steps and open questions

Takeaway

- Codebase understanding improves when runtime evidence, file references, and documentation prompts work together.